

A CUDA Convolutional Network for CIFAR-10 Image Classification

Report focused on image-processing aspects of the implemented net

26 June 2026

1. Abstract

This report analyses a CUDA/C implementation of a shallow convolutional neural network trained on CIFAR-10. The network receives a low-resolution RGB image, applies learned spatial convolution filters, introduces non-linearity with ReLU, reduces local spatial variation with max pooling, and classifies the resulting feature vector with a fully connected softmax head. The run used all 50,000 training samples and all 10,000 test samples. Over ten epochs, the test accuracy increased from 32.85% to 51.75%, while the test loss decreased from 1.9355 to 1.3848. The final train-test gap was small, approximately 0.51 percentage points, which indicates that the principal limitation is model capacity and representation rather than overfitting.

Contents

1. Abstract	1
2. Dataset: CIFAR-10 as an image-processing benchmark	2
3. Input representation and preprocessing	2
4. Network architecture	4
5. CUDA implementation and training loop	5
6. Results after ten epochs	7
7. Timing analysis	8
8. Image-processing interpretation of the learned pipeline	9
8.1. Convolution as learned spatial filtering	9
8.2. ReLU as non-linear feature selection	9
8.3. Max pooling as local invariance and downsampling	9
8.4. Fully connected layer as global template classifier	10
8.5. Softmax and prediction	10
9. Limitations	11
10. Recommendations	11
11. Conclusion	11
12. References	11
13. Appendix: reproducibility notes	12

2. Dataset: CIFAR-10 as an image-processing benchmark

CIFAR-10 is a standard small-image benchmark from the University of Toronto. The official dataset description states that it contains 60,000 RGB colour images of size 32×32 , organized into ten mutually exclusive object classes with 6,000 images per class. The standard split contains 50,000 training images and 10,000 test images. The official binary and Python/Matlab layouts store each image as 3072 channel values: 1024 red, 1024 green, and 1024 blue pixels. See the official CIFAR page.

The dump confirms the standard balanced split used in this run:

La-bel	Class	Train sam-ples	Test sam-ples
0	airplane	5000	1000
1	automobile	5000	1000
2	bird	5000	1000
3	cat	5000	1000
4	deer	5000	1000
5	dog	5000	1000
6	frog	5000	1000
7	horse	5000	1000
8	ship	5000	1000
9	truck	5000	1000

From an image-processing perspective, CIFAR-10 is demanding because the objects are semantically rich but spatially tiny. A 32×32 image contains only 1024 spatial samples per channel, so object boundaries, texture, background context, and color cues are heavily compressed. Good performance therefore depends on filters that can capture local edge/color patterns while preserving enough spatial layout for object-level discrimination.

3. Input representation and preprocessing

The implementation indexes images from class directories, decodes PNG/JPEG images with the `stb_image` open source library, forces three RGB channels, resizes each image to 32×32 using bilinear interpolation and normalizes pixel values from integer byte values to floating point values in $[0, 1]$. The output tensor layout is NCHW:

- **N**: batch size,
- **C**: channel count (3 for RGB),
- **H**: height (32 pixels),
- **W**: width (32 pixels).

The preprocessing can be summarized as:

```
file image -> RGB decode -> bilinear resize to 32x32 -> divide by 255 -> NCHW tensor
```

The source constants fix the image shape and class count:

```
// dataset.h
#define NUM_CLASSES    10
#define IMAGE_WIDTH    32
#define IMAGE_HEIGHT   32
#define IMAGE_CHANNELS  3
```

The dataset loader confirms the class-directory assumption and the CIFAR-10 class order used by the program:

```
// dataset.c
const char *CIFAR_CLASS_NAMES[NUM_CLASSES] = {
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
};
```

```
// Labels are subfolders in dataset directory, each named after the class
for (int label = 0; label < NUM_CLASSES; label++) {
    if (!load_class_directory(dataset, root_dir,
                             CIFAR_CLASS_NAMES[label], label)) return 0;
}
```

The actual pixel preprocessing is implemented as bilinear interpolation followed by NCHW storage and normalization:

```
// dataset.c, shortened excerpt
// Bilinear resizing and normalization to RGB [0,1] in NCHW format
static void resize_bilinear(const unsigned char *src,
                           int src_w, int src_h, float *dst) {
    const int dst_w = IMAGE_WIDTH;
    const int dst_h = IMAGE_HEIGHT;
    const int channels = IMAGE_CHANNELS;
    const int dst_image_size = dst_w * dst_h;

    for (int dy = 0; dy < dst_h; dy++) {
        float src_y = ((float)dy + 0.5f) *
                      ((float)src_h / (float)dst_h) - 0.5f;
        int y0 = (int)floorf(src_y);
        float wy = src_y - (float)y0;
        if (y0 < 0) { y0 = 0; wy = 0.0f; }
        int y1 = y0 + 1;
        if (y1 >= src_h) y1 = src_h - 1;

        for (int dx = 0; dx < dst_w; dx++) {
            float src_x = ((float)dx + 0.5f) *
                          ((float)src_w / (float)dst_w) - 0.5f;
            int x0 = (int)floorf(src_x);
            float wx = src_x - (float)x0;
            if (x0 < 0) { x0 = 0; wx = 0.0f; }
            int x1 = x0 + 1;
            if (x1 >= src_w) x1 = src_w - 1;

            for (int c = 0; c < channels; c++) {
                int idx00 = (y0 * src_w + x0) * channels + c;
                int idx01 = (y0 * src_w + x1) * channels + c;
                int idx10 = (y1 * src_w + x0) * channels + c;
                int idx11 = (y1 * src_w + x1) * channels + c;

                float p00 = (float)src[idx00];
                float p01 = (float)src[idx01];
                float p10 = (float)src[idx10];
                float p11 = (float)src[idx11];

                float top = p00 * (1.0f - wx) + p01 * wx;
                float bot = p10 * (1.0f - wx) + p11 * wx;
                float val = top * (1.0f - wy) + bot * wy;

                int dst_idx = c * dst_image_size + dy * dst_w + dx;
                dst[dst_idx] = val / 255.0f;
            }
        }
    }
}
```

This is a minimal and reproducible preprocessing pipeline. It is adequate for verifying a custom CUDA network, but it isn't optimal in terms of performance because it lacks common data augmentation and normalization techniques. CIFAR-10 is already 32×32 in its native form, so resizing is useful only if the data have been converted into image files with uncertain dimensions. The implementation also does not apply per-channel mean/std normalization, random crop, horizontal flip, color jitter, or cutout-style augmentation. These omissions matter

because CIFAR-10 classification benefits from invariance to small translations, mirror transformations, lighting changes, and background clutter.

4. Network architecture

The network is a compact CNN with a single learned convolutional feature extraction stage followed by a fully connected classifier. It can be described as a shallow LeNet-style architecture:

Stage	Operation	Output tensor	Image-processing role
Input	RGB image	$3 \times 32 \times 32$	Three color channels at low spatial resolution.
Convolution	32 kernels, 5×5 , stride 1, padding 2	$32 \times 32 \times 32$	Learns local spatial/color filters while preserving image size.
Activation	ReLU	$32 \times 32 \times 32$	Keeps positive filter responses and introduces nonlinearity.
Pooling	2×2 max pool, stride 2	$32 \times 16 \times 16$	Downsamples feature maps and adds local translation robustness.
Flatten	Vectorization	8192	Converts spatial feature maps to classifier input.
Classifier	Fully connected + bias	10 logits	Maps extracted features to class scores.
Output	Softmax + argmax	10 probabilities + prediction	Produces normalized class confidence and predicted label.

The architecture constants are compiled from the header file:

```
// kernels.h
#define KERNEL_H 5
#define KERNEL_W 5
#define KERNEL_COUNT 32

#define PADDING_Y 2
#define PADDING_X 2
#define STRIDE_Y 1
#define STRIDE_X 1

#define BATCH_SIZE 16

#define OUTPUT_H ((INPUT_H + 2 * PADDING_Y - KERNEL_H) / STRIDE_Y + 1)
#define OUTPUT_W ((INPUT_W + 2 * PADDING_X - KERNEL_W) / STRIDE_X + 1)

#define POOL_SIZE 2
#define POOL_STRIDE 2
#define POOL_OUTPUT_H ((OUTPUT_H - POOL_SIZE) / POOL_STRIDE + 1)
#define POOL_OUTPUT_W ((OUTPUT_W - POOL_SIZE) / POOL_STRIDE + 1)

#define FLATTEN_SIZE (KERNEL_COUNT * POOL_OUTPUT_H * POOL_OUTPUT_W)
#define EPOCHS 10
#define FIXED_SEED 123
```

The following table summarizes the parameter count and role of each learnable component. The convolutional kernels are the only parameters in the feature extractor, while the fully connected weights and bias are the only parameters in the classifier:

Parameter group	Shape	Count	Comment
-----------------	-------	-------	---------

Convolution kernels	$32 \times 3 \times 5 \times 5$	2,400	Local color-spatial filters.
Fully connected weights	8192×10	81,920	Classifier matrix after flattening.
Fully connected bias	10	10	One bias per class.
Total implemented		84,330	97.1% of parameters are in the FC weights.

It is worth noting that the flatten size of 8192 is large relative to the network depth. This means that most parameters and most measured forward time reside in the classifier, which is a limitation of the current architecture. Also note that the convolutional kernels are the only learnable parameters in the feature extractor, so the network's ability to extract useful features is limited by the number of kernels and their size.

5. CUDA implementation and training loop

The forward pass is decomposed into five timed GPU stages: convolution, ReLU, max pooling, fully connected plus bias, and softmax plus prediction. The convolution kernel uses shared memory tiling. The classifier uses a shared-memory matrix multiplication kernel to compute logits from the flattened pooled tensor.

The main training loop exposes the host-side learning protocol: shuffle the training index, load and preprocess a batch, transfer it to the GPU, run forward propagation, run backpropagation, compute loss, and accumulate accuracy.

```
// main.cu, shortened excerpt
const float learningRate = 0.001f;
const float lambda = 1e-4f;

for (int epoch = 0; epoch < EPOCHS; epoch++) {
    shuffle_dataset(&train);

    for (int batch = 0; batch < trainBatches; batch++) {
        int startIndex = batch * BATCH_SIZE;
        int loaded = load_batch(&train, startIndex,
                                BATCH_SIZE, h_input, h_labels);
        if (loaded != BATCH_SIZE) continue;

        cudaMemcpy(d_input, h_input,
                   (size_t)inputElements * sizeof(float),
                   cudaMemcpyHostToDevice);
        cudaMemcpy(d_labels, h_labels,
                   (size_t)BATCH_SIZE * sizeof(int),
                   cudaMemcpyHostToDevice);

        LeNet_forward(d_input, d_labels, cnn,
                      collectTiming ? forwardTiming : NULL);
        LeNet_backward(d_input, d_labels, cnn,
                       learningRate, lambda, NULL);

        float batchLoss = compute_batch_loss(cnn->d_softmax_output,
                                              d_labels, d_loss,
                                              h_loss, BATCH_SIZE);
    }
}
```

The forward pass maps directly to the architecture table:

```
// lenet.cu, shortened forward-pass excerpt
convolutionSharedKernel<<<convGridDim, convBlockDim, sharedMemSize>>>(
    d_input, cnn->d_kernels, cnn->d_conv_output,
    BATCH_SIZE, INPUT_CHANNELS, INPUT_SIZE, KERNEL_SIZE,
    KERNEL_COUNT, OUTPUT_SIZE, PADDING, STRIDE);
```

```

cudaMemcpy(cnn->d_activation, cnn->d_conv_output,
            convTotalElements * sizeof(float), cudaMemcpyDeviceToDevice);
reluActivationKernel<<<reluGridSize, blockSize>>>(
    cnn->d_activation, convTotalElements);

maxPoolingKernel<<<poolGridDim, poolBlockDim>>>(
    cnn->d_activation, cnn->d_pooling_output,
    BATCH_SIZE, KERNEL_COUNT, OUTPUT_SIZE,
    POOL_SIZE, POOL_OUTPUT_SIZE, POOL_STRIDE);

matrixMultiplySharedKernel<<<mmGridDim, mmBlockDim>>>(
    cnn->d_pooling_output, cnn->d_fc_weights, cnn->d_logits,
    BATCH_SIZE, FLATTEN_SIZE, NUM_CLASSES);
addBiasKernel<<<biasGridDim, biasBlockDim>>>(
    cnn->d_logits, cnn->d_fc_bias, BATCH_SIZE, NUM_CLASSES);

softmaxKernel<<<predGridSize, predBlockSize>>>(
    cnn->d_logits, cnn->d_softmax_output, BATCH_SIZE, NUM_CLASSES);
getPredictionsKernel<<<predGridSize, predBlockSize>>>(
    cnn->d_softmax_output, cnn->d_predictions, BATCH_SIZE, NUM_CLASSES);

```

The backward pass follows the usual cross-entropy/softmax training path:

Backward stage	Purpose
Softmax cross-entropy gradient	Computes (softmax - one_hot(label)) / batch_size for the logits.
FC gradients	Computes fully connected weight, bias, and input gradients.
Max-pooling backward	Routes gradients only to maximum locations selected by the forward pooling window.
ReLU backward	Suppresses gradients where pre-ReLU convolution responses were non-positive.
Convolution weight gradient	Reduces gradients over batch, spatial position, and input channel dimensions.
L2 regularization and SGD	Adds ridge-style weight decay and applies SGD updates.

The source-level implementation of those stages is visible in the following backpropagation excerpt:

```

// lenet.cu, shortened backward-pass excerpt
softmaxCrossEntropyKernel<<<logitsGrid, blockSize>>>(
    cnn->d_softmax_output, d_labels, cnn->d_d_logits,
    BATCH_SIZE, NUM_CLASSES);

fcWeightGradientKernel<<<fcWeightGrid, fcWeightBlock>>>(
    cnn->d_pooling_output, cnn->d_d_logits, cnn->d_d_fc_weights,
    BATCH_SIZE, FLATTEN_SIZE, NUM_CLASSES);
fcBiasGradientKernel<<<biasGrid, blockSize>>>(
    cnn->d_d_logits, cnn->d_d_fc_bias, BATCH_SIZE, NUM_CLASSES);
fcInputGradientKernel<<<fcInputGrid, fcInputBlock>>>(
    cnn->d_d_logits, cnn->d_fc_weights, cnn->d_d_pooling_output,
    BATCH_SIZE, FLATTEN_SIZE, NUM_CLASSES);

cudaMemset(cnn->d_d_activation, 0,
            convTotalElements * sizeof(float));
maxPoolingBackwardKernel<<<poolGridDim, poolBlockDim>>>(
    cnn->d_activation, cnn->d_d_pooling_output, cnn->d_d_activation,
    BATCH_SIZE, KERNEL_COUNT, OUTPUT_SIZE,
    POOL_SIZE, POOL_OUTPUT_SIZE, POOL_STRIDE);

reluBackwardKernel<<<reluGrid, blockSize>>>(
    cnn->d_conv_output, cnn->d_d_activation,

```

```

cnn->d_d_conv_output, convTotalElements);

conv2WeightGradientReduceKernel<<<convWGrid, blockSize,
                                blockSize * sizeof(float)>>>(
    d_input, cnn->d_d_conv_output, cnn->d_d_kernels,
    BATCH_SIZE, INPUT_CHANNELS, INPUT_SIZE, INPUT_SIZE,
    KERNEL_COUNT, KERNEL_SIZE, KERNEL_SIZE,
    OUTPUT_SIZE, OUTPUT_SIZE, PADDING, PADDING, STRIDE, STRIDE);

ridgeL2GradientKernel<<<fcUpdateGrid, blockSize>>>(
    cnn->d_d_fc_weights, cnn->d_fc_weights, fcWeights, lambda);
sgdUpdateKernel<<<fcUpdateGrid, blockSize>>>(
    cnn->d_fc_weights, cnn->d_d_fc_weights, learningRate, fcWeights);

```

The run used a fixed seed of 123, batch size 16, learning rate 0.001, L2 coefficient 1e-4, and ten epochs. Each epoch used 3125 training batches and 625 test batches, matching exactly 50,000 training and 10,000 test images with batch size 16.

6. Results after ten epochs

The epoch-level results show steady optimization over all ten epochs.

Epoch	Train loss	Train acc.	Test loss	Test acc.
1	2.0985	24.65%	1.9355	32.85%
2	1.8700	34.99%	1.8339	35.11%
3	1.7682	38.83%	1.7208	41.26%
4	1.6844	41.88%	1.6530	43.03%
5	1.6107	44.31%	1.5793	44.98%
6	1.5461	46.71%	1.5197	47.51%
7	1.4916	48.59%	1.4678	48.87%
8	1.4466	50.07%	1.4307	50.08%
9	1.4096	51.30%	1.4026	50.73%
10	1.3812	52.26%	1.3848	51.75%

The curves below visualize the same data.

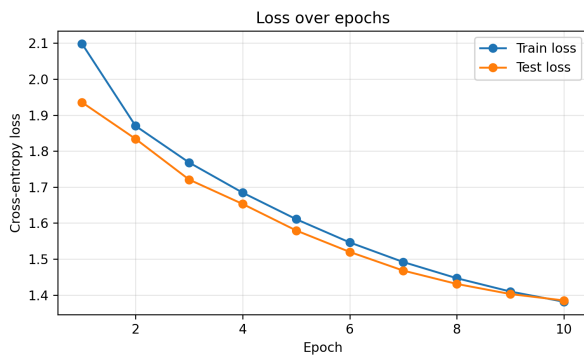


Figure 1: Cross-entropy loss over epochs.

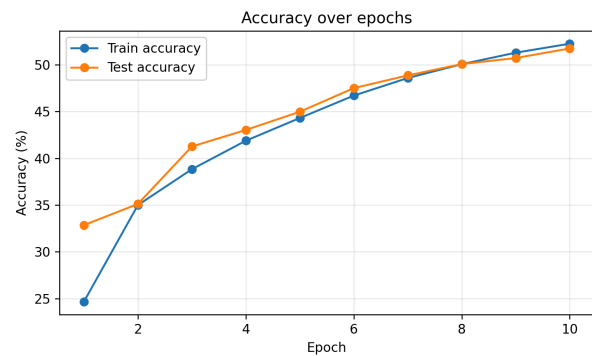


Figure 2: Classification accuracy over epochs.

The most important observations are:

- Training and test accuracy improve monotonically at the epoch-summary level, ending at 52.26% and 51.75% respectively.
- The final train-test accuracy gap is only 0.51 percentage points, so the network is not strongly overfitting.
- The final test accuracy is far above the 10% random-choice baseline, but below what deeper CIFAR-10 CNNs can achieve. A state-of-the-art LeNet5, which is the classifier used for this shallow net, underperforms on CIFAR-10, achieving $75.35\% \pm 0.55\%$ success rate after more epochs.

- The convergence curve has not saturated completely after ten epochs; later epochs, a learning-rate schedule, or a stronger optimizer could plausibly improve the result.

7. Timing analysis

The kernel-level timing was measured after every ten batches and at the last batch of each phase. In total, the parsed timing set contains 3760 forward timing lines and 3130 training backward timing lines. The average measured forward pass on training batches is 8.164 ms; the average measured forward pass on test batches is 8.037 ms. The average measured backward pass on logged training batches is 2.193 ms. Thus, the measured training compute path is approximately 10.36 ms per logged batch, excluding data loading and host-to-device transfer overheads.

Forward component	Train mean ms	Train share	Test mean ms	Test share
Conv	0.722	8.84%	0.727	9.05%
ReLU	0.012	0.14%	0.013	0.16%
Max pool	0.023	0.28%	0.024	0.29%
FC + bias	7.372	90.30%	7.239	90.08%
Softmax + prediction	0.035	0.43%	0.034	0.43%

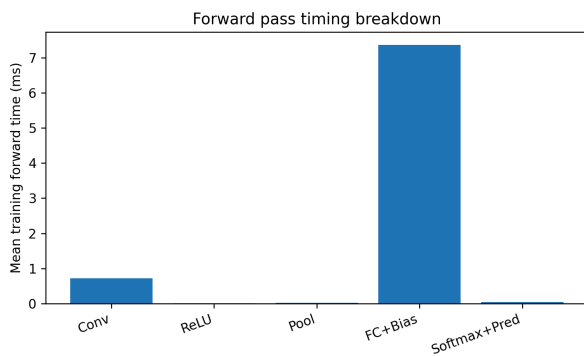


Figure 3: Mean training forward-pass timing by component.

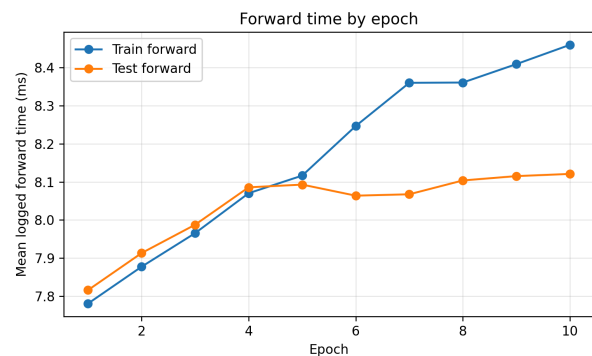


Figure 4: Mean logged forward time per epoch.

The striking result is that FC+Bias accounts for about 90% of measured forward time, even though the convolution has more theoretical multiply-add work. This indicates that the bottleneck is implementation-level rather than purely arithmetic. Likely contributors include memory access patterns, small matrix dimensions, launch overhead, and the custom matrix multiplication kernel. Replacing the custom fully connected path with cuBLAS GEMM, using a deeper convolutional frontend that reduces spatial dimensions before flattening, or replacing the large flatten-to-FC head with global average pooling would make the implementation more balanced.

For reference, the mean forward time per epoch was:

Epoch	Train forward ms	Test forward ms
1	7.781	7.816
2	7.878	7.913
3	7.966	7.988
4	8.070	8.086
5	8.116	8.093
6	8.246	8.064
7	8.360	8.067
8	8.360	8.104
9	8.409	8.115
10	8.459	8.121

The gradual increase in logged training forward time across epochs is small but visible. It is not caused by changing tensor shapes, so it is more likely due to runtime variability, timing overhead, GPU state, or memory/cache effects than the mathematical model itself.

8. Image-processing interpretation of the learned pipeline

8.1. Convolution as learned spatial filtering

The first layer performs 32 learned 5×5 convolutions over RGB input. In classical image processing, such filters can be interpreted as local spatial operators responding to color contrast, edges, corners, and small texture primitives. The padding value of 2 preserves the 32×32 spatial grid, which is useful because CIFAR-10 objects already occupy few pixels.

The CUDA convolution explicitly computes each output value as a local weighted sum over input channel, kernel row, and kernel column. The indexing below also shows the NCHW layout used by the implementation.

```
// kernels.cu, shortened convolution inner loop
for (int c = 0; c < inputChannels; c++) {
    for (int ky = 0; ky < kernelSize; ky++) {
        for (int kx = 0; kx < kernelSize; kx++) {
            int shared_y = ty * stride + ky;
            int shared_x = tx * stride + kx;

            float in_val = sharedInput[
                c * tileSizeWithPadding * tileSizeWithPadding +
                shared_y * tileSizeWithPadding + shared_x];

            float kernel_val = kernels[
                k * inputChannels * kernelSize * kernelSize +
                c * kernelSize * kernelSize + ky * kernelSize + kx];

            sum += in_val * kernel_val;
        }
    }
}

output[b * kernelCount * outputSize * outputSize +
    k * outputSize * outputSize + out_y * outputSize + out_x] = sum;
```

A limitation is that one convolutional layer cannot build a hierarchy of increasingly abstract image features. A deeper design would allow early filters to detect oriented edges and color blobs, middle filters to combine them into parts and textures, and later filters to assemble object-level cues. The current model jumps from one low-level filter bank directly to a large classifier.

8.2. ReLU as non-linear feature selection

ReLU keeps positive filter responses and suppresses negative ones. This makes the representation sparse and non-linear. In image-processing terms, it acts like a half-wave rectifier applied to filter responses, retaining strong evidence for the learned local patterns.

```
// kernels.cu
__global__ void reluActivationKernel(float *data, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size)
        data[idx] = fmaxf(0.0f, data[idx]);
}
```

8.3. Max pooling as local invariance and downsampling

The 2×2 max-pooling stage halves the spatial size from 32×32 to 16×16 . This reduces feature-map storage by a factor of four and makes the classifier less sensitive to one-pixel shifts. The cost is loss of precise spatial detail. On CIFAR-10, this is a reasonable tradeoff because object category is more important than exact pixel alignment, but aggressive pooling in a very shallow network can also discard useful localization information.

```
// kernels.cu, shortened max-pooling excerpt
float maxVal = -FLT_MAX;

for (int dy = 0; dy < poolSize; dy++) {
    for (int dx = 0; dx < poolSize; dx++) {
        int in_y = in_y_base + dy;
        int in_x = in_x_base + dx;

        if (in_y < inputSize && in_x < inputSize) {
            float value = input[b * channels * inputSize * inputSize +
                               c * inputSize * inputSize +
                               in_y * inputSize + in_x];
            maxVal = fmaxf(maxVal, value);
        }
    }
}

output[b * channels * outputSize * outputSize +
       c * outputSize * outputSize + out_y * outputSize + out_x] = maxVal;
```

8.4. Fully connected layer as global template classifier

After pooling, the 32 feature maps are flattened to 8192 values. The classifier then learns ten global templates over this flattened feature vector. This makes the output decision depend on absolute feature-map positions. That can help when object placement is consistent, but it is weaker when objects translate, rotate, or appear against cluttered backgrounds.

The source uses a tiled matrix multiplication kernel for this stage, followed by an explicit bias-add kernel:

```
// kernels.cu, shortened matrix multiplication excerpt
__shared__ float sA[TILE_SIZE][TILE_SIZE];
__shared__ float sB[TILE_SIZE][TILE_SIZE];

for (int tile = 0; tile < numTiles; tile++) {
    int aCol = tile * TILE_SIZE + threadIdx.x;
    int bRow = tile * TILE_SIZE + threadIdx.y;

    sA[threadIdx.y][threadIdx.x] =
        (row < A_rows && aCol < A_cols) ? A[row * A_cols + aCol] : 0.0f;
    sB[threadIdx.y][threadIdx.x] =
        (bRow < A_cols && col < B_cols) ? B[bRow * B_cols + col] : 0.0f;

    __syncthreads();
    for (int k = 0; k < TILE_SIZE; k++)
        sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
    __syncthreads();
}

if (row < A_rows && col < B_cols)
    C[row * B_cols + col] = sum;
```

8.5. Softmax and prediction

The final image-classification step converts logits to probabilities with a max-shifted softmax and chooses the maximum-probability class.

```
// kernels.cu, shortened softmax excerpt
float max_val = -FLT_MAX;
for (int i = 0; i < num_classes; i++)
    max_val = fmaxf(max_val, input[batch_idx * num_classes + i]);

float sum = 0.0f;
for (int i = 0; i < num_classes; i++) {
    output[batch_idx * num_classes + i] =
        expf(input[batch_idx * num_classes + i] - max_val);
    sum += output[batch_idx * num_classes + i];
}
```

```

}

for (int i = 0; i < num_classes; i++)
    output[batch_idx * num_classes + i] /= sum;

```

9. Limitations

The implementation is clear and useful as a CUDA CNN baseline, but the following image-processing and learning limitations explain the modest final accuracy:

- The network has only one convolutional layer, so its learned representation is mostly low-level.
- There is no data augmentation, so the model is not explicitly trained for common image transformations.
- There is no per-channel standardization, which may slow optimization and make color statistics harder to learn.
- The flatten size is large relative to the network depth, causing most parameters and most measured forward time to reside in the classifier.
- No confusion matrix or per-class accuracy is logged, so the report cannot identify which visual categories are most confused.
- The training loop uses plain SGD with a fixed learning rate; no momentum, adaptive optimizer, or learning-rate schedule is used.

10. Recommendations

A stronger image-processing CNN for CIFAR-10 should keep the efficient CUDA structure but change the representation:

1. Add convolutional blocks: for example, (conv -> ReLU -> conv -> ReLU -> pool) repeated two or three times.
2. Add per-channel normalization using training-set RGB means and standard deviations.
3. Add random crop with padding, horizontal flip, and light color augmentation.
4. Reduce the fully connected bottleneck with additional pooling, global average pooling, or a smaller hidden layer.
5. Use momentum SGD or Adam and a learning-rate schedule.
6. Log a confusion matrix and per-class precision/recall to connect performance errors to image content, such as animals versus vehicles.
7. Consider cuBLAS for the fully connected matrix multiply and fuse simple elementwise kernels where possible.

11. Conclusion

The implemented network is a compact CUDA CNN for CIFAR-10. It correctly follows the image-processing pattern of local filtering, non-linear activation, local pooling, and global classification. The experiment demonstrates learning: test accuracy rises from 32.85% after epoch 1 to 51.75% after epoch 10. The small train-test gap suggests that the model is not mainly limited by overfitting; rather, it is limited by shallow feature extraction, minimal preprocessing, and a classifier-heavy architecture. The best next step is to preserve the CUDA implementation style while deepening the convolutional feature extractor and reducing dependence on the flatten-to-FC classifier.

12. References

1. Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. “The CIFAR-10 and CIFAR-100 datasets.” University of Toronto. Dataset page.
2. Alex Krizhevsky. “Learning Multiple Layers of Features from Tiny Images.” Technical report, 2009.
3. Rafael C. Gonzalez and Richard E. Woods. “Digital Image Processing”, 4th edition. Pearson, 2018.
4. Yuval Meir, Itamar Ben-Noam, Yarden Tzach, Shiri Hodassman & Ido Kanter. “Learning on tree architectures outperforms a convolutional feedforward network”, Nature 2023/01/30 DOI: 10.1038/s41598-023-27986-6.

13. Appendix: reproducibility notes

- Dataset root expected by the executable: `dataset/train/<class-name>` and `dataset/test/<class-name>`.
- Class names in the source code: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- Main training constants: `BATCH_SIZE = 16`, `EPOCHS = 10`, `FIXED_SEED = 123`, `learningRate = 0.001`, `lambda = 1e-4`.
- Tensor sizes from the run: input $3 \times 32 \times 32$, convolution output $32 \times 32 \times 32$, pool output $32 \times 16 \times 16$, flatten size 8192, output classes 10.
- The timing tables in this report are parsed from logged CUDA events and do not include all CPU-side dataset decoding, preprocessing, and host-device transfer costs.